RA1 e RA2 — Online Retail

Gastão Eduardo e Gustavo Schott · Sistemas de Informação, 6º período · Professora Marina de Lara

No **RA1**, preparamos o histórico da loja. No **RA2**, mostramos seis eventos fictícios chegando em três momentos.

No Google Colab

1. Envie este arquivo `.ipynb`.
2. Selecione **Ambiente de execução > Executar tudo**.
3. Aguarde todas as células terminarem. A primeira execução instala as dependências e baixa a base.

Ambiente

Pandas organiza os dados em tabelas. Apache Spark processa os arquivos que chegam no RA2. As versões usadas e os resultados aparecem nas células.

```python
import os
import sys
import subprocess
import importlib.util
from pathlib import Path

try:
    import google.colab
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

if IN_COLAB:
    subprocess.run(
        [sys.executable, "-m", "pip", "install", "-q",
         "pandas==2.3.3", "openpyxl==3.1.5", "pyspark==3.5.7"], check=True
    )
    java17 = Path("/usr/lib/jvm/java-17-openjdk-amd64")
    if not java17.exists():
        subprocess.run(["apt-get", "update", "-qq"], check=True)
        subprocess.run(["apt-get", "install", "-y", "-qq",
                        "openjdk-17-jre-headless"], check=True)
    os.environ["JAVA_HOME"] = str(java17)
    os.environ["PATH"] = str(java17 / "bin") + os.pathsep + os.environ["PATH"]

import json
import hashlib
import platform
import urllib.request
import zipfile
import pandas as pd
import numpy as np
import pyspark
from datetime import datetime, timezone
from IPython.display import display

ROOT = Path.cwd()
for relative in ["data/raw", "data/processed", "evidence", "data/stream_runs"]:
    (ROOT / relative).mkdir(parents=True, exist_ok=True)

java_info = subprocess.run(["java", "-version"], capture_output=True, text=True, check=True)
java_version = next(line for line in java_info.stderr.splitlines() if "version" in line.lower())
print("Python:", platform.python_version())
print("pandas:", pd.__version__, "| PySpark:", pyspark.__version__)
print("Java:", java_version)
```

```
Python: 3.12.15
pandas: 2.3.3 | PySpark: 3.5.7
Java: openjdk version "17.0.20" 2026-07-21
```

RA1 — Ingestão e preparação

Problema e objetivo

Uma loja precisa de registros organizados para analisar suas transações. O objetivo aqui é conferir a base Online Retail e preparar registros positivos de mercadorias, de todos os países e durante o período disponível.

Perguntas para análises futuras:

1. Como o valor dos registros positivos varia por dia?
2. Quais produtos têm maior quantidade registrada?
3. No RA2, como os valores por minuto mudam quando chegam novos eventos?

No RA1, preparamos os dados; não respondemos às duas primeiras perguntas nem calculamos lucro ou vendas líquidas. Uma linha é um item da transação, não necessariamente um pedido inteiro.

RA1.2 Fonte e ingestão

Fonte: Chen (2015), *Online Retail*, UCI Machine Learning Repository. Página da base · DOI 10.24432/C5BW33 · Licença CC BY 4.0.

A base contém transações de uma loja do Reino Unido, de 2010 a 2011, com preços em GBP. O arquivo é um Excel com oito colunas. O código baixa o ZIP oficial, confere seu hash e lê a planilha `Online Retail`.

```
SOURCE_URL = "https://archive.ics.uci.edu/static/public/352/online%2Bretail.zip"
EXPECTED_ZIP_SHA256 = "f5385cbb54bbebf7196389109c6b0621faab0c304e3702548165e71c84aede8b"
archive_path = ROOT / "data/raw/online_retail_uci.zip"
source_path = ROOT / "data/raw/Online Retail.xlsx"

if not archive_path.exists():
    request = urllib.request.Request(SOURCE_URL, headers={"User-Agent": "AcademicDataProject/1.0"})
    with urllib.request.urlopen(request, timeout=180) as response:
        payload = response.read()
    if hashlib.sha256(payload).hexdigest() != EXPECTED_ZIP_SHA256:
        raise ValueError("A versão da fonte mudou; revisar o arquivo antes de continuar.")
    archive_path.write_bytes(payload)

archive_hash = hashlib.sha256(archive_path.read_bytes()).hexdigest()
assert archive_hash == EXPECTED_ZIP_SHA256, "ZIP diferente da versão verificada"
with zipfile.ZipFile(archive_path) as archive:
    source_bytes = archive.read("Online Retail.xlsx")
if not source_path.exists():
    source_path.write_bytes(source_bytes)
assert hashlib.sha256(source_path.read_bytes()).digest() == hashlib.sha256(source_bytes).digest()

raw = pd.read_excel(source_path, sheet_name="Online Retail", engine="openpyxl")
source_evidence = {
    "download_url": SOURCE_URL, "license": "CC BY 4.0",
    "checked_at_utc": datetime.now(timezone.utc).isoformat(),
    "zip_sha256": archive_hash,
    "xlsx_sha256": hashlib.sha256(source_path.read_bytes()).hexdigest(),
    "rows": len(raw), "columns": len(raw.columns),
}
(ROOT / "evidence/source_verified.json").write_text(
    json.dumps(source_evidence, indent=2, ensure_ascii=False), encoding="utf-8")
print("Registros carregados:", len(raw), "| Colunas:", len(raw.columns))
print("SHA-256 do XLSX:", source_evidence["xlsx_sha256"])
display(raw.head(3)[["InvoiceNo", "StockCode", "Quantity", "UnitPrice"]])
display(raw.head(3)[["InvoiceDate", "Description", "Country"]])
```

```
Registros carregados: 541909 | Colunas: 8
SHA-256 do XLSX: 43465a06f2ccf7c8b5bd2892bc7defb52f97487934fe93b16ae4c3936424676d
```

	InvoiceNo	StockCode	Quantity	UnitPrice
0	536365	85123A	6	2.55
1	536365	71053	6	3.39
2	536365	84406B	8	2.75

	InvoiceDate	Description	Country
0	2010-12-01 08:26:00	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
1	2010-12-01 08:26:00	WHITE METAL LANTERN	United Kingdom
2	2010-12-01 08:26:00	CREAM CUPID HEARTS COAT HANGER	United Kingdom

RA1.3 Qualidade dos dados

As oito colunas são: `InvoiceNo` (transação), `StockCode` e `Description` (item), `Quantity`, `InvoiceDate`, `UnitPrice`, `CustomerID` e `Country`.

Uma fatura pode aparecer em várias linhas. As contagens de qualidade podem se sobrepor; não devem ser somadas para calcular exclusões. O balanço sem sobreposição aparece na próxima etapa.

```
description_text = raw.Description.astype("string")
quality = pd.DataFrame([
    ("Q1", "Description ausente", int(raw.Description.isna().sum())),
    ("Q2", "CustomerID ausente", int(raw.CustomerID.isna().sum())),
    ("Q3", "Repetições extras nas oito colunas", int(raw.duplicated().sum())),
    ("Q4", "InvoiceNo com prefixo C", int(raw.InvoiceNo.astype(str).str.upper().str.startswith("C").sum())),
    ("Q5", "Quantity menor ou igual a zero", int(raw.Quantity.le(0).sum())),
    ("Q6", "UnitPrice menor ou igual a zero", int(raw.UnitPrice.le(0).sum())),
    ("Q7", "Description com espaços nas pontas",
     int(description_text.ne(description_text.str.strip()).fillna(False).sum())),
    ("Q8", "InvoiceDate inválida", int(pd.to_datetime(raw.InvoiceDate, errors="coerce").isna().sum())),
], columns=["ID", "Verificação", "Quantidade"])
quality["Percentual da base"] = (quality.Quantidade / len(raw) * 100).round(2)
display(quality)
print("Denominador de todos os percentuais:", len(raw), "linhas brutas")
print("Período:", raw.InvoiceDate.min(), "até", raw.InvoiceDate.max())
display(pd.DataFrame({"Coluna": raw.columns, "Tipo observado": raw.dtypes.astype(str).values}))
display(raw[["Quantity", "UnitPrice"]].describe(percentiles=[0.5, 0.99]).T)
quality.to_csv(ROOT / "evidence/quality_checks.csv", index=False)
```

	ID	Verificação	Quantidade	Percentual da base
0	Q1	Description ausente	1454	0.27
1	Q2	CustomerID ausente	135080	24.93
2	Q3	Repetições extras nas oito colunas	5268	0.97
3	Q4	InvoiceNo com prefixo C	9288	1.71
4	Q5	Quantity menor ou igual a zero	10624	1.96
5	Q6	UnitPrice menor ou igual a zero	2517	0.46
6	Q7	Description com espaços nas pontas	113452	20.94
7	Q8	InvoiceDate inválida	0	0.00

```
Denominador de todos os percentuais: 541909 linhas brutas
Período: 2010-12-01 08:26:00 até 2011-12-09 12:50:00
```

	Coluna	Tipo observado
0	InvoiceNo	object
1	StockCode	object
2	Description	object
3	Quantity	int64
4	InvoiceDate	datetime64[ns]
5	UnitPrice	float64
6	CustomerID	float64
7	Country	object

	count	mean	std	min	50%	99%	max
Quantity	541909.0	9.552250	218.081158	-80995.00	3.00	100.0	80995.0
UnitPrice	541909.0	4.611114	96.759853	-11062.06	2.08	18.0	38970.0

Clientes sem ID foram mantidos: ainda é possível usar o item e o horário. Linhas idênticas foram sinalizadas e mantidas porque a fonte não fornece uma chave única de linha. Quantidades e preços positivos extremos também foram mantidos; a loja atende atacadistas. Cancelamentos e valores não positivos precisam ser interpretados, não tratados automaticamente como erro.

RA1.4 Preparação

- Textos padronizados; datas e números convertidos; identificadores tratados como texto.
- Campos essenciais inválidos, cancelamentos, quantidades ou preços não positivos e códigos de frete, taxas, ajustes ou vouchers ficam separados.
- Clientes ausentes, repetições, extremos positivos e códigos raros de produtos são preservados.
- Cada linha mantém sua origem no Excel (`SourceRow`) e, se separada, recebe o primeiro motivo aplicável.

A função abaixo contém as regras usadas na preparação.

```python
"""Preparação para estudar registros positivos de mercadorias, não vendas líquidas."""
import numpy as np
import pandas as pd


COLUMNS = ["InvoiceNo", "StockCode", "Description", "Quantity", "InvoiceDate",
           "UnitPrice", "CustomerID", "Country"]


def prepare_data(raw):
    """Retorna dados preparados, linhas fora do recorte e balanço das decisões.

    SourceRow aponta para a linha do Excel original (cabeçalho na linha 1).
    Nenhuma linha é descartada sem aparecer no arquivo de exclusões.
    """
    missing = set(COLUMNS) - set(raw.columns)
    if missing:
        raise ValueError(f"Colunas obrigatórias ausentes: {sorted(missing)}")
    data = raw[COLUMNS].copy().reset_index(drop=True)
    data.insert(0, "SourceRow", np.arange(2, len(data) + 2))
    for column in ["InvoiceNo", "StockCode", "Description", "CustomerID", "Country"]:
        data[column] = data[column].astype("string").str.strip().replace("", pd.NA)
    # O Excel armazena IDs de cliente numéricos; o sufixo .0 vem da leitura com nulos.
    data["CustomerID"] = data["CustomerID"].str.replace(r"\.0$", "", regex=True)
    data["InvoiceDate"] = pd.to_datetime(data["InvoiceDate"], errors="coerce")
    data["Quantity"] = pd.to_numeric(data["Quantity"], errors="coerce")
    data["UnitPrice"] = pd.to_numeric(data["UnitPrice"], errors="coerce")

    reasons = pd.Series(pd.NA, index=data.index, dtype="string")

    def exclude(mask, reason):
        # A primeira regra aplicável evita contar a mesma linha em dois motivos.
        reasons.loc[reasons.isna() & mask.fillna(False)] = reason

    exclude(data["InvoiceNo"].str.upper().str.startswith("C"), "cancellation")
    required = ["InvoiceNo", "StockCode", "InvoiceDate", "Quantity", "UnitPrice", "Country"]
    invalid = data[required].isna().any(axis=1)
    invalid |= ~np.isfinite(data["Quantity"]) | ~np.isfinite(data["UnitPrice"])
    invalid |= data["Quantity"].mod(1).ne(0)
    exclude(invalid, "invalid_required_fields")
    exclude(data["Quantity"].le(0), "non_positive_quantity")
    exclude(data["UnitPrice"].le(0), "non_positive_price")
    # As descrições observadas identificam estes códigos como frete, taxas,
    # descontos, ajustes manuais, amostras ou vouchers. Códigos raros de produtos
    # (por exemplo DCGSSBOY e PADS) são mantidos; formato sozinho não prova erro.
    other_codes = {"POST", "DOT", "C2", "M", "D", "BANK CHARGES", "AMAZONFEE",
                   "CRUK", "B", "S"}
    code = data["StockCode"].str.upper()
    exclude(code.isin(other_codes) | code.str.startswith("GIFT_"),
            "non_merchandise_code")
    exclude(data["Description"].isna(), "missing_description")

    clean = data.loc[reasons.isna()].copy()
    clean["Quantity"] = clean["Quantity"].astype("Int64")
    clean["UnitPrice"] = clean["UnitPrice"].astype("Float64")
    # Clientes ausentes, repetições exatas e extremos são preservados intencionalmente.
    excluded = data.loc[reasons.notna()].copy()
    excluded["ExclusionReason"] = reasons.loc[reasons.notna()]
    summary = {
        "input_rows": int(len(raw)), "prepared_rows": int(len(clean)),
        "excluded_rows": int(len(excluded)), "input_columns": int(raw.shape[1]),
        "prepared_columns": int(clean.shape[1]),
        "exclusions_by_reason": {k: int(v) for k, v in reasons.value_counts().items()},
        "missing_customers_kept": int(clean["CustomerID"].isna().sum()),
        "exact_duplicate_rows_kept": int(clean[COLUMNS].duplicated().sum()),
        "source_rows_unique": bool(clean["SourceRow"].is_unique),
        "date_min": str(clean["InvoiceDate"].min()),
        "date_max": str(clean["InvoiceDate"].max()),
```

```
    }
    assert len(clean) + len(excluded) == len(raw)
    assert not set(clean["SourceRow"]).intersection(excluded["SourceRow"])
    return clean, excluded, summary
```

```
prepared, excluded, preparation_summary = prepare_data(raw)
prepared.to_csv(ROOT / "data/processed/sales_prepared.csv", index=False)
excluded.to_csv(ROOT / "data/processed/excluded_records.csv", index=False)
(ROOT / "evidence/preparation_summary.json").write_text(
    json.dumps(preparation_summary, indent=2, ensure_ascii=False), encoding="utf-8")

reason_labels = {
    "cancellation": "Cancelamento identificado pelo prefixo C",
    "invalid_required_fields": "Campo essencial inválido",
    "non_positive_quantity": "Quantidade não positiva, após regras anteriores",
    "non_positive_price": "Preço não positivo, após regras anteriores",
    "non_merchandise_code": "Frete, taxas, ajustes, amostras ou voucher",
    "missing_description": "Descrição ausente após regras anteriores",
}
exclusion_table = pd.DataFrame([
    (reason_labels[key], int((excluded.ExclusionReason == key).sum()))
    for key in reason_labels
], columns=["Primeiro motivo aplicável", "Linhas separadas"])
display(exclusion_table)
print("Balanço:", len(raw), "=", len(prepared), "preparadas +", len(excluded), "separadas")
display(excluded[["SourceRow", "InvoiceNo", "StockCode", "ExclusionReason"]].head(6))
```

	Primeiro motivo aplicável	Linhas separadas
0	Cancelamento identificado pelo prefixo C	9288
1	Campo essencial inválido	0
2	Quantidade não positiva, após regras anteriores	1336
3	Preço não positivo, após regras anteriores	1181
4	Frete, taxas, ajustes, amostras ou voucher	2343
5	Descrição ausente após regras anteriores	0

```
Balanço: 541909 = 527761 preparadas + 14148 separadas
```

	SourceRow	InvoiceNo	StockCode	ExclusionReason
45	47	536370	POST	non_merchandise_code
141	143	C536379	D	cancellation
154	156	C536383	35004C	cancellation
235	237	C536391	22556	cancellation
236	238	C536391	21984	cancellation
237	239	C536391	21983	cancellation

RA1.5 Resultado e validação

O arquivo preparado contém **527.761 linhas**; **14.148** ficaram separadas com justificativa. O balanço é `541.909 = 527.761 + 14.148`.

`sales_prepared.csv` contém os registros do recorte. `excluded_records.csv` mantém as linhas separadas e seus motivos. As verificações confirmam campos essenciais, quantidades e preços positivos, origem única e balanço completo.

```
essential = ["InvoiceNo", "StockCode", "Description", "Quantity", "InvoiceDate", "UnitPrice", "Country"]
assert prepared[essential].notna().all().all()
assert prepared.Quantity.gt(0).all() and prepared.Quantity.mod(1).eq(0).all()
assert prepared.UnitPrice.gt(0).all()
assert not prepared.InvoiceNo.str.upper().str.startswith("C").any()
assert prepared.SourceRow.is_unique
assert len(prepared) + len(excluded) == len(raw)
assert not set(prepared.SourceRow).intersection(excluded.SourceRow)

loaded = pd.read_csv(ROOT / "data/processed/sales_prepared.csv",
    dtype={"InvoiceNo": "string", "StockCode": "string", "CustomerID": "string",
           "Quantity": "Int64", "SourceRow": "int64"}, parse_dates=["InvoiceDate"])
assert len(loaded) == len(prepared)
assert loaded.SourceRow.tolist() == prepared.SourceRow.tolist()
assert loaded.CustomerID.isna().sum() == prepared.CustomerID.isna().sum()

comparison = pd.DataFrame([
    ("Linhas", len(raw), len(prepared)),
    ("Colunas", raw.shape[1], prepared.shape[1]),
    ("Descrições ausentes", int(raw.Description.isna().sum()), int(prepared.Description.isna().sum())),
    ("Clientes ausentes", int(raw.CustomerID.isna().sum()), int(prepared.CustomerID.isna().sum())),
    ("Repetições extras nos 8 atributos", int(raw.duplicated().sum()),
     int(prepared[COLUMNS].duplicated().sum())),
], columns=["Medida", "Após ingestão", "Após preparação"])
display(comparison)
print("Todas as verificações de consistência e releitura passaram.")
comparison.to_csv(ROOT / "evidence/before_after.csv", index=False)
```

	Medida	Após ingestão	Após preparação
0	Linhas	541909	527761
1	Colunas	8	9
2	Descrições ausentes	1454	0
3	Clientes ausentes	135080	131421
4	Repetições extras nos 8 atributos	5268	5221

```
Todas as verificações de consistência e releitura passaram.
```

Limites e reprodução

A base cobre uma loja e um período histórico específico. Não representa todo o comércio online. O recorte não é receita líquida nem lucro: não ligamos cada cancelamento à venda correspondente.

O notebook contém o processo de ingestão e preparação. A documentação de entrega acompanha o template da disciplina. Fonte: Chen (2015), *Online Retail*, UCI Machine Learning Repository, DOI 10.24432/C5BW33, CC BY 4.0.

RA2 — Spark Structured Streaming

Objetivo

Pergunta: como a soma do valor dos itens por minuto muda conforme chegam novos eventos? Usamos o cenário do enunciado com uma fonte estática e chegadas simuladas.

A única análise é `Quantity × UnitPrice` por janela de um minuto. Os eventos são fictícios; os códigos e nomes de dois produtos vêm do conjunto preparado no RA1.

RA2.2 Eventos simulados

São seis eventos fictícios em três arquivos, dois por chegada. Horários, quantidades e preços foram criados para a demonstração. A hora no evento (`event_time`) determina a janela; a hora em que o arquivo chega ao Spark pode ser diferente.

Resultados esperados: chegada 1 → **25 GBP**; chegada 2 → **55 e 20 GBP**; chegada 3 → **55, 30 e 10 GBP**. Soma final: **95 GBP**.

```
import uuid
from decimal import Decimal
from pyspark.sql import SparkSession, functions as F, types as T

os.environ.setdefault("SPARK_LOCAL_IP", "127.0.0.1")
os.environ["PYSPARK_PYTHON"] = sys.executable
spark = (SparkSession.builder.master("local[2]")
    .appName("RA2OnlineRetail")
    .config("spark.ui.enabled", "false")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.sql.session.timeZone", "UTC")
    .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

run_id = uuid.uuid4().hex
run_dir = ROOT / "data/stream_runs" / run_id
input_dir = run_dir / "input"
staging_dir = run_dir / "staging"
input_dir.mkdir(parents=True)
staging_dir.mkdir()
query_name = "sales_" + run_id

catalog = loaded[["StockCode", "Description"]].drop_duplicates("StockCode").head(2)
products = catalog.to_dict("records")
event_specs = [
    ("SIM001", "10:00:05", 2, 10.00, 0),
    ("SIM002", "10:00:30", 1, 5.00, 1),
    ("SIM003", "10:00:45", 3, 10.00, 0),
    ("SIM004", "10:01:00", 4, 5.00, 1),
    ("SIM005", "10:01:20", 1, 10.00, 0),
    ("SIM006", "10:02:05", 2, 5.00, 1),
]
events = [
    {"event_id": event_id, "event_time": "2026-10-15T" + clock + "Z",
     "StockCode": products[product]["StockCode"],
     "Description": products[product]["Description"],
     "Quantity": quantity, "UnitPrice": price, "synthetic": True}
    for event_id, clock, quantity, price, product in event_specs
]
assert len({event["event_id"] for event in events}) == 6
assert all(event["StockCode"] in set(catalog.StockCode) for event in events)
display(pd.DataFrame(events)[["event_id", "event_time", "StockCode", "Quantity", "UnitPrice"]])
print("Eventos artificiais; códigos e descrições vêm do conjunto preparado no RA1.")
```

```
NOTE: Picked up JDK_JAVA_OPTIONS: -Djava.io.tmpdir=/workspace/.work/container-tmp

NOTE: Picked up JDK_JAVA_OPTIONS: -Djava.io.tmpdir=/workspace/.work/container-tmp

Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

26/10/20 17:08:55 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-j
ava classes where applicable

26/10/20 17:08:56 WARN MacAddressUtil: Failed to find a usable hardware address from the network interfaces; using
random bytes: a6:01:1d:6a:04:53:84:62
```

	event_id	event_time	StockCode	Quantity	UnitPrice
0	SIM001	2026-10-15T10:00:05Z	85123A	2	10.0
1	SIM002	2026-10-15T10:00:30Z	71053	1	5.0
2	SIM003	2026-10-15T10:00:45Z	85123A	3	10.0
3	SIM004	2026-10-15T10:01:00Z	71053	4	5.0
4	SIM005	2026-10-15T10:01:20Z	85123A	1	10.0
5	SIM006	2026-10-15T10:02:05Z	71053	2	5.0

Eventos artificiais; códigos e descrições vêm do conjunto preparado no RA1.

RA2.3 Janela e consulta

A janela **tumbling** dura um minuto e não se sobrepõe à seguinte. O início entra e o fim não: `[10:00, 10:01)`. Por isso, o evento das 10:01:00 pertence à janela que começa às 10:01.

`readStream` acompanha novos arquivos, `window` agrupa pelo horário do evento e `sum` soma os valores. `writeStream` atualiza a tabela da demonstração em memória. Ela não é armazenamento permanente de produção.

```
schema = T.StructType([
    T.StructField("event_id", T.StringType(), False),
    T.StructField("event_time", T.TimestampType(), False),
    T.StructField("StockCode", T.StringType(), False),
    T.StructField("Description", T.StringType(), False),
    T.StructField("Quantity", T.IntegerType(), False),
    T.StructField("UnitPrice", T.DecimalType(12, 2), False),
    T.StructField("synthetic", T.BooleanType(), False),
])
incoming = (spark.readStream.schema(schema).option("mode", "FAILFAST")
            .option("maxFilesPerTrigger", 1).json(str(input_dir)))
assert incoming.isStreaming
window_totals = (incoming
    .withColumn("item_value", F.col("Quantity") * F.col("UnitPrice"))
    .groupBy(F.window("event_time", "1 minute"))
    .agg(F.sum("item_value").alias("total_gbp"), F.count("event_id").alias("events")))
query = (window_totals.writeStream.format("memory").queryName(query_name)
    .outputMode("complete")
    .option("checkpointLocation", str(run_dir / "checkpoint"))
    .trigger(processingTime="1 second").start())
print("Consulta ativa:", query.isActive, "| DataFrame streaming:", incoming.isStreaming)
snapshots = []
```

```
Consulta ativa: True | DataFrame streaming: True
```

```
def publish_batch(number, batch_events, expected):
    # Grava fora da pasta observada e move o arquivo completo de uma vez.
    staged = staging_dir / f"batch_{number}.json"
    target = input_dir / staged.name
    if target.exists():
        raise ValueError("Esta chegada já foi publicada; reinicie o RA2 para repetir.")
    payload = "\n".join(json.dumps(event, ensure_ascii=False) for event in batch_events) + "\n"
    staged.write_text(payload, encoding="utf-8")
    staged.rename(target)
    arrived_at = datetime.now(timezone.utc).isoformat()
    query.processAllAvailable()
    if query.exception() is not None:
        raise RuntimeError(str(query.exception()))

    rows = (spark.table(query_name).orderBy("window.start")
            .select(F.date_format("window.start", "HH:mm:ss").alias("start"),
                    F.date_format("window.end", "HH:mm:ss").alias("end"),
                    "total_gbp", "events").collect())
    observed = {row["start"]: (str(row["total_gbp"]), row["events"]) for row in rows}
    assert observed == expected, f"Resultado diferente do esperado: {observed}"
    data_progress = [p for p in query.recentProgress if p["numInputRows"] > 0]
    progress = data_progress[-1]
    assert progress["numInputRows"] == len(batch_events)
    snapshot = {"arrival": number, "arrived_at_utc": arrived_at,
                "spark_batch_id": progress["batchId"],
                "new_rows": progress["numInputRows"], "windows": observed}
    snapshots.append(snapshot)
    print(f"Chegada {number}: {len(batch_events)} novos registros | batchId={progress['batchId']}")
    print("Recebidos em UTC:", arrived_at)
    display(pd.DataFrame([row.asDict() for row in rows]).rename(columns={
        "start": "Início", "end": "Fim exclusivo", "total_gbp": "Total GBP", "events": "Eventos"}))
    print("Resultado conferido com a conta manual.")
```

Chegada 1: `20 + 5 = 25 GBP` na janela 10:00.

```
publish_batch(1, events[:2], {"10:00:00": ("25.00", 2)})
```

```
[Stage 0:>                                                          (0 + 1) / 1]

[Stage 1:>                                                          (0 + 2) / 2]

[Stage 1:==============================>                          (1 + 1) / 2]


Chegada 1: 2 novos registros | batchId=0
Recebidos em UTC: 2026-10-20T17:09:07.087525+00:00
```

	Início	Fim exclusivo	Total GBP	Eventos
0	10:00:00	10:01:00	25.00	2

```
Resultado conferido com a conta manual.
```

Chegada 2: a janela 10:00 passa a 55 GBP; às 10:01 começa outra com 20 GBP.

```
publish_batch(2, events[2:4], {"10:00:00": ("55.00", 3), "10:01:00": ("20.00", 1)})
```

```
Chegada 2: 2 novos registros | batchId=1
Recebidos em UTC: 2026-10-20T17:09:16.116906+00:00
```

	Início	Fim exclusivo	Total GBP	Eventos
0	10:00:00	10:01:00	55.00	3
1	10:01:00	10:02:00	20.00	1

```
Resultado conferido com a conta manual.
```

Chegada 3: a janela 10:01 passa a 30 GBP e a janela 10:02 começa com 10 GBP.

```
publish_batch(3, events[4:], {"10:00:00": ("55.00", 3), "10:01:00": ("30.00", 2), "10:02:00": ("10.00", 1)})
```

```
Chegada 3: 2 novos registros | batchId=2
Recebidos em UTC: 2026-10-20T17:09:19.500480+00:00
```

	Início	Fim exclusivo	Total GBP	Eventos
0	10:00:00	10:01:00	55.00	3
1	10:01:00	10:02:00	30.00	2
2	10:02:00	10:03:00	10.00	1

```
Resultado conferido com a conta manual.
```

```
assert sum(snapshot["new_rows"] for snapshot in snapshots) == 6
assert len({snapshot["spark_batch_id"] for snapshot in snapshots}) == 3
assert sum(Decimal(value[0]) for value in snapshots[-1]["windows"].values()) == Decimal("95.00")
(ROOT / "evidence/streaming_snapshots.json").write_text(
    json.dumps(snapshots, ensure_ascii=False, indent=2), encoding="utf-8")
print("Validação final: 3 chegadas distintas, 6 eventos, 3 janelas e total de 95.00 GBP.")
query.stop()
print("Consulta encerrada:", not query.isActive)
spark.stop()
```

```
Validação final: 3 chegadas distintas, 6 eventos, 3 janelas e total de 95.00 GBP.
Consulta encerrada: True
```

RA2.5 Resultado e limites

As três chegadas processaram dois eventos cada. Os totais observados conferem com a conta manual. São resultados da simulação, não conclusões sobre as vendas históricas.

A demonstração usa uma janela e uma agregação. Não implementa Kafka, armazenamento durável, watermark ou tratamento de falhas de produção.

```
execution_record = {
    "executed_at_utc": datetime.now(timezone.utc).isoformat(),
    "execution_type": "registro gerado ao alcançar a última célula do notebook",
    "python": platform.python_version(), "pandas": pd.__version__,
    "pyspark": pyspark.__version__, "java": java_version,
    "source_sha256": source_evidence["xlsx_sha256"],
    "prepared_rows": len(prepared), "excluded_rows": len(excluded),
    "ra1_checks_passed": True, "ra2_checked_arrivals": len(snapshots),
    "ra2_checked_events": sum(item["new_rows"] for item in snapshots),
    "synthetic_total_gbp": "95.00",
}
(ROOT / "evidence/execution_record.json").write_text(
    json.dumps(execution_record, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps(execution_record, ensure_ascii=False, indent=2))
```

```
{
  "executed_at_utc": "2026-10-20T17:09:24.835719+00:00",
  "execution_type": "registro gerado ao alcançar a última célula do notebook",
  "python": "3.12.15",
  "pandas": "2.3.3",
  "pyspark": "3.5.7",
  "java": "openjdk version \"17.0.20\" 2026-07-21",
  "source_sha256": "43465a06f2ccf7c8b5bd2892bc7defb52f97487934fe93b16ae4c3936424676d",
  "prepared_rows": 527761,
  "excluded_rows": 14148,
  "ra1_checks_passed": true,
  "ra2_checked_arrivals": 3,
  "ra2_checked_events": 6,
  "synthetic_total_gbp": "95.00"
}
```